

Здравствуй, мир конкурентности в C++!

В этой главе

- Что подразумевается под конкурентностью и многопоточностью.
- Зачем использовать конкурентность и многопоточность в ваших приложениях.
- История поддержки конкурентности в C++.
- Как выглядит простая многопоточная программа на C++.

Для пользователей C++ наступили интересные времена. Спустя 13 лет после опубликования стандарта C++ (это произошло в 1998 году) Комитет по стандартизации C++ полностью переработал язык и поддерживаемую его библиотеку. Новый стандарт C++, обозначаемый C++11 или C++0x, был опубликован в 2011 году и принес с собой ряд изменений, сделавших работу с языком более простой и продуктивной. Комитет также обязался разработать новую «модель передач» выпусков, согласно которой новый стандарт C++ будет публиковаться каждые три года. Пока вышли две такие публикации: C++14 в 2014 году и C++17 — в 2017-м, а также несколько технических спецификаций с описаниями расширений стандарта C++.

Из самых значимых новшеств в стандарте C++11 стоит отметить поддержку многопоточных программ. Впервые Комитет признал существование многопоточных приложений на этом языке и добавил в библиотеку средства для написания многопоточных приложений. Появилась возможность писать на C++ многопоточные

программы с гарантированным поведением, не полагаясь на платформенные расширения. Все это было очень вовремя, так как программисты, стремясь повысить производительность приложений, все чаще стали обращать внимание на конкурентность в целом и многопоточное программирование в частности. Стандарты C++14 и C++17 обеспечивают поддержку создания многопоточных программ на C++, но уже при наличии соответствующих технических спецификаций. Есть техническая спецификация для расширений, призванных выполнять конкурентные вычисления, и еще одна спецификация для параллельных вычислений (включена только в C++17).

В этой книге рассматриваются создание конкурентных программ на C++ с использованием нескольких потоков, а также свойства языка C++ и библиотечные средства, обеспечивающие соответствующие возможности. Сначала я объясню, что подразумевается под конкурентностью и многопоточностью и что может побудить вас использовать конкурентность в своих приложениях. После краткого обзора причин, по которым вы можете отказаться от ее применения в приложениях, будет вкратце рассмотрена поддержка конкурентности в C++, а в завершение главы приведу пример простой конкурентной программы на языке C++. Читатели, имеющие опыт разработки многопоточных приложений, могут пропустить первые разделы. В последующих главах мы рассмотрим более сложные примеры и подробно изучим средства библиотеки. В конце книги будут детально описаны все возможности стандартной библиотеки C++, позволяющие реализовать многопоточность и конкурентность.

Итак, что же понимается под *конкурентностью* (concurrency) и *многопоточностью* (multithreading)?

1.1. Что такое конкурентность

На самом элементарном уровне конкурентность — это одновременное выполнение двух и более отдельных задач. За примерами из обычной жизни далеко ходить не надо: мы можем одновременно идти и разговаривать, а еще выполнять разные действия левой и правой руками; каждый из нас живет своей жизнью независимо от других людей — пока я буду плавать, вы можете смотреть футбол и т. д.

1.1.1. Конкурентность в компьютерных системах

Когда речь заходит о конкурентности в контексте компьютеров, подразумевается отдельно взятая система, выполняющая несколько независимых задач в параллельном, а не последовательном (одна за другой) режиме. Эта идея не нова. Многозадачные операционные системы, позволяющие настольному компьютеру запускать одновременно несколько приложений с помощью переключения задач были обычным явлением на протяжении многих лет, как и высокопроизводительные серверы с несколькими процессорами, обеспечивавшие действительно параллельные

вычисления. Новым является массовое распространение компьютеров, способных по-настоящему выполнять несколько задач в параллельном режиме, а не создавать иллюзию этого процесса.

Исторически сложилось так, что у большинства настольных компьютеров был один процессор с одним вычислительным блоком или ядром, да и сегодня для многих настольных компьютеров ничего в этом смысле не изменилось. Такая машина в конкретный момент времени выполняет только одну задачу, но может переключаться между задачами множество раз в секунду. При выполнении небольшой части одной задачи, затем небольшой части другой и т. д. создается впечатление, что задачи решаются одновременно. Это называется *переключением задач*. Имея в виду такие системы, мы все же говорим о *конкурентности*, поскольку переключение задач происходит так быстро, что невозможно сказать, в какой момент задача может быть приостановлена из-за переключения процессора на выполнение другой задачи. Переключение между задачами создает иллюзию конкурентности как для пользователя, так и для самих приложений. Поскольку это всего лишь иллюзия таких вычислений, поведение приложений при выполнении в среде переключения задач с одним процессором может немного отличаться от их поведения при выполнении в среде с истинным параллелизмом. В частности, неверные допущения о модели памяти (рассматриваемой в главе 5) могут не проявляться в такой среде. Более подробно этот вопрос обсуждается в главе 10.

Компьютеры с несколькими процессорами в течение многих лет использовались в качестве серверов и для решения вычислительных задач, для которых требуется высокопроизводительное оборудование, а компьютеры на базе процессоров с несколькими ядрами на одном кристалле (многоядерные процессоры) теперь получают все большее распространение в качестве настольных. Независимо от того, имеют компьютеры несколько процессоров или несколько ядер в процессоре (или и то и другое), они способны выполнять несколько задач в параллельном режиме. Это называется *аппаратной конкурентностью*.

На рис. 1.1 схематически показан сценарий работы компьютера с двумя задачами, каждая из которых разделена на десять одинаковых этапов. На двухъядерной машине (имеющей два вычислительных ядра) каждая задача может выполняться в собственном ядре. На одноядерном компьютере, переключающем задачи, этапы задач чередуются. Кроме того, они немного разнесены по времени, что показано серыми разделительными полосами, более толстыми по сравнению с разделительными полосами на схеме для двухъядерной машины. Чтобы выполнить чередование, система при каждом переходе от одной задачи к другой должна *переключать контекст*, на что требуется время. Чтобы переключить контекст, операционная система должна сохранить состояние центрального процессора и указатель команд для выполняемой в данный момент задачи, определить, на какую задачу переключиться, и перезагрузить состояние центрального процессора для задачи, на которую происходит переключение. В таком случае центральному процессору, возможно, придется загружать в кэш-память инструкции и данные для новой задачи, что может помешать ему выполнять любые другие инструкции и привести к еще большей задержке.

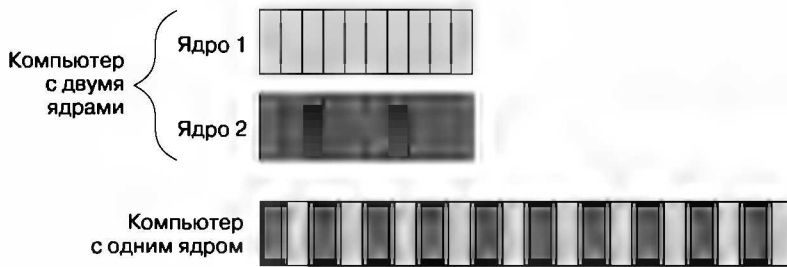


Рис. 1.1. Два подхода к конкурентности: параллельное выполнение на компьютере с двумя ядрами и переключение задач на одноядерной машине

Хотя возможности конкурентности нагляднее проявляются в многопроцессорных или многоядерных системах, некоторые процессоры могут выполнять несколько потоков и на одном ядре. Следует учитывать очень важный фактор — количество *аппаратных потоков*, то есть то количество независимых задач, которое оборудование может выполнить действительно одновременно. В то же время в системе с истинным аппаратным параллелизмом количество задач может превышать количество ядер, и тогда будет применяться механизм переключения задач. Например, на обычном настольном компьютере могут выполняться сотни задач, работающих в фоновом режиме, даже если машина номинально простаивает. Именно благодаря переключению эти задачи могут выполняться параллельно и можно одновременно открыть текстовый процессор, компилятор, редактор и браузер (или любую комбинацию приложений). На рис. 1.2 показано такое переключение между четырьмя задачами на двухъядерном компьютере, опять же для идеализированного сценария, где задачи делятся на этапы одинаковой длительности. На практике в силу многих причин деление не будет таким равномерным, а переключение — таким распределенным. Некоторые из этих причин рассматриваются в главе 8 при изучении факторов, влияющих на производительность конкурентного кода.



Рис. 1.2. Переключение при решении четырех задач на двух ядрах

Все технические приемы, функции и классы, рассматриваемые в данной книге, можно задействовать независимо от того, на каком компьютере выполняется ваше приложение — с одним одноядерным процессором или с несколькими многоядерными процессорами. И неважно, как именно реализуется конкурентность — путем переключения задач или за счет настоящего аппаратного параллелизма. Но, наверное, понятно, что способ обеспечения конкурентности в вашем приложении может зависеть от доступного оборудования. Соответствующие вопросы

рассматриваются в главе 8 при изучении задач, связанных с разработкой на C++ конкурентного кода.

1.1.2. Подходы к конкурентности

Представьте двух программистов, работающих вместе над одним программным проектом. Если они находятся в разных кабинетах, то могут работать спокойно, не мешая друг другу и пользуясь каждый своим набором справочников. Но вот общаться им непросто. Вместо того чтобы развернуться лицом друг к другу и поговорить, они вынуждены звонить по телефону, писать по электронной почте или же вставать и идти в другой кабинет к коллеге. Кроме того, приходится нести затраты на аренду двух помещений и приобретение нескольких экземпляров справочников.

А теперь представьте, что разработчики сидят в одном кабинете. Они могут свободно общаться, обсуждать проект приложения, рисовать схемы на бумаге или доске, обмениваться своими идеями.

Теперь нужно содержать только один кабинет, и зачастую достаточно будет только одного набора ресурсов. В то же время разработчикам может быть труднее сосредоточиться, и вполне реальны проблемы с общими ресурсами («Куда опять подевался справочник?»).

Эти два способа организации труда разработчиков позволяют проиллюстрировать два основных подхода к конкурентности. Каждый разработчик представляет поток, а каждый кабинет — процесс. Первый подход состоит в том, чтобы иметь несколько однопоточных процессов, что похоже на работу каждого разработчика в собственном кабинете, а второй подход предполагает наличие нескольких потоков в одном процессе, что похоже на работу двух разработчиков в одном помещении.

На такой основе можно составлять какие угодно комбинации и иметь несколько процессов, часть которых являются многопоточными, а часть — однопоточными, но действуют по одним и тем же принципам. Теперь кратко рассмотрим эти два подхода к конкурентности в приложениях.

Конкурентность с использованием нескольких процессов

Первый способ применения параллелизма в приложениях — разделение приложения на несколько конкурентных процессов с одним потоком. Например, так происходит при одновременном запуске браузера и текстового процессора. В дальнейшем отдельные процессы могут пересылать сообщения по всем обычным каналам межпроцессного обмена данными (используя сигналы, сокеты, файлы, конвейеры и т. д.) (рис. 1.3). Один из недостатков способа заключается в том, что либо такую связь между процессами сложно установить, либо она слишком медленно работает, либо наблюдается и то и другое. Так происходит из-за того, что операционные системы при обмене данными между процессами обычно предпринимают множество защитных мер, чтобы один процесс не мог случайно изменить данные, принадлежащие другому процессу. Еще одним недостатком является то, что выполнение нескольких процессов сопряжено с накладными расходами: на

запуск процесса требуется время, операционная система для управления им должна выделять внутренние ресурсы и т. д.

Но не все так плохо: операционные системы с дополнительной защитой обычно обеспечивают взаимодействие процессов и механизмов обмена данными более высокого уровня, а это значит, что конкурентный код может быть проще создавать с процессами, а не с потоками. И действительно, в среде, подобной той, что предусмотрена для языка программирования Erlang (www.erlang.org/), в качестве основного механизма конкурентности весьма эффективно используются именно процессы.

У использования отдельных процессов для реализации конкурентности есть и дополнительное преимущество — процессы можно запускать на разных компьютерах, входящих в одну сеть. Хотя это и увеличивает затраты на обмен данными, в качественно спроектированной системе способ может стать экономически эффективным для повышения степени конкурентности и увеличения производительности.

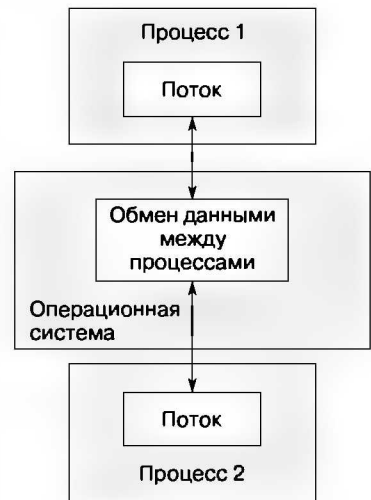


Рис. 1.3. Обмен данными между двумя одновременно запущенными процессами

Конкурентность с применением нескольких потоков

Альтернативный подход к конкурентности состоит в выполнении нескольких потоков в одном процессе. Потоки очень похожи на облегченные процессы: каждый из них реализуется независимо от других и может выполнять собственную последовательность инструкций. Но все потоки в процессе используют общее адресное пространство, и непосредственный доступ к основной части данных можно получить из всех потоков. При этом глобальные переменные остаются глобальными, а указатели или ссылки на объекты или данные можно передавать между потоками. Для процессов можно организовать доступ к разделяемой памяти, но эта функция трудно поддается настройке и управлению, потому что адреса размещения в памяти одних и тех же элементов не обязательно будут одинаковыми для разных процессов. На рис. 1.4 показаны два потока внутри процесса, обменивающиеся данными через общую память.

Наличие общего адресного пространства и отсутствие защиты данных между потоками существенно сокращают издержки, связанные с использованием нескольких потоков, по сравнению с издержками при выполнении нескольких процессов, так как снижается нагрузка на операционную систему. Но гибкость, обусловленная применением общей памяти, сопряжена и с затратами: если доступ к данным получают несколько потоков, программист должен убедиться, что представление данных,

видимых каждым потоком, согласовано при каждом обращении к этим данным. Вопросы, связанные с обменом данными между потоками, используемые для их решения инструменты, а также рекомендации, которых следует придерживаться во избежание возникновения проблем, рассматриваются в книге повсеместно, особенно в главах 3–5 и 8. Проблемы вполне решаемы при условии, что при создании кода вы будете осмотрительны, а это означает, что обмен данными между потоками следует тщательно продумать.

Низкие издержки на запуск потоков внутри процесса и обмен данными между ними стали причиной популярности этого подхода во всех распространенных языках программирования, включая C++, даже несмотря на потенциальные проблемы, связанные с разделением памяти. Кроме того, стандарт C++ не предусматривает встроенной поддержки обмена данными между процессами, из-за чего приложения с несколькими процессами придется полагаться на API, зависящие от платформы. Поэтому данная книга посвящена исключительно конкурентности на основе многопоточности, и в дальнейшем при любом упоминании конкурентности предполагается, что она реализуется использованием нескольких потоков.

Для многопоточного кода часто употребляется другое слово — «параллелизм». Давайте уточним, в чем здесь различия.

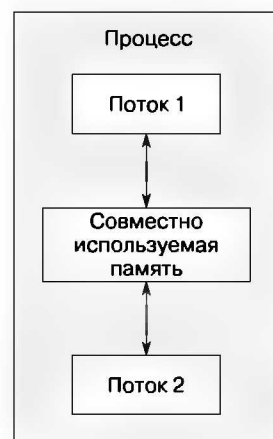


Рис. 1.4. Обмен данными между двумя потоками, запущенными одновременно в одном процессе

1.1.3. Сравнение конкурентности и параллелизма

Для многопоточного кода значения понятий «конкурентность» и «параллелизм» в значительной степени пересекаются. Действительно, для многих они означают одно и то же. Разница в первую очередь заключается в нюансах, фокусе и намерениях. Оба термина относятся к одновременному выполнению нескольких задач с использованием доступного оборудования, но параллелизм более явно ориентирован на производительность. О *параллелизме* говорят, когда главной заботой становится использование имеющегося оборудования для повышения производительности массовой обработки данных, тогда как при *конкурентности* основная забота — разделение задач, требующих одновременного решения, или оперативность реагирования. Такая двойственность до сих пор существует, и значения этих двух понятий все еще существенно совпадают, но знать, в чем их различие, полезно при обсуждении рассматриваемых вопросов. В этой книге будут встречаться примеры использования обоих понятий.

Уточнив, что именно подразумевается под конкурентностью и параллелизмом, давайте выясним, зачем нужна конкурентность в приложениях.

1.2. Зачем используется конкурентность

Существует две основные причины применения параллелизма в приложении: разделение задач и повышение производительности. Возьму на себя смелость заявить, что это почти *единственные* причины использования параллелизма, все остальное, если приглядеться, сводится либо к одному, либо к другому (или даже и к тому и к другому, за исключением таких причин, как «потому что мне так хочется»).

1.2.1. Конкурентность для разделения неотложных задач

Разделение задач при создании программных средств — это практически всегда весьма рациональный подход. Сгруппировав связанные фрагменты кода и обособив несвязанные, проще понять и протестировать свои программы, к тому же так можно снизить вероятность появления ошибок. Конкурентность можно задействовать для разделения функциональных областей, даже если операции в этих отдельных областях должны выполняться одновременно. Без явного использования конкурентности пришлось бы либо создавать среду переключения задач, либо в ходе операции постоянно вызывать не связанные с этой операцией области кода.

Рассмотрим приложение, интенсивно обрабатывающее данные и имеющее пользовательский интерфейс, например приложение DVD-плеера для настольного компьютера. У него в основном два набора обязанностей. Оно должно не только считывать данные с диска, декодировать изображения и звук и своевременно отправлять их на монитор и звуковую карту, не допуская сбоев в воспроизведении DVD, но и принимать данные от пользователя, например, когда он щелкает на кнопках паузы, возврата в меню или даже выхода. В одном потоке во время воспроизведения приложение должно через равные промежутки времени проверять, что вводит пользователь, сменявая код воспроизведения DVD с кодом интерфейса пользователя. Используя многопоточность для разделения этих задач, можно избавиться от тесного переплетения кода пользовательского интерфейса и кода воспроизведения DVD: один поток может обрабатывать пользовательский интерфейс, а другой — воспроизведение DVD. Между ними должно быть организовано взаимодействие, например, когда пользователь нажимает кнопку паузы, но теперь эти моменты взаимодействия напрямую связаны с поставленной задачей.

Это создает иллюзию отзывчивости, поскольку поток пользовательского интерфейса обычно способен тут же ответить на запрос пользователя, даже если во время передачи запроса потоку, выполняющему работу, ответ должен отображать курсор в виде значка занятости или сообщение с просьбой подождать. Точно так же отдельные потоки часто используются для запуска задач, которые должны выполняться непрерывно в фоновом режиме, например отслеживания изменений файловой системы в приложении поиска на настольном компьютере. Как правило, подобное использование потоков существенно упрощает логику в каждом из них, поскольку взаимодействие между потоками может быть ограничено строго опреде-

ленными моментами, не переплетаясь с логикой различных задач. В данном случае количество потоков не зависит от количества доступных ядер центрального процессора, так как разделение на потоки основано на концентуальной схеме, а не на попытке увеличения пропускной способности.

1.2.2. Конкурентность для повышения производительности: параллелизм задач и данных

История многопроцессорных систем насчитывает десятилетия, но до недавнего времени они встречались в основном в суперкомпьютерах, мейнфреймах и крупных серверных системах. Производители микросхем все чаще отдают предпочтение многоядерным разработкам с 2, 4, 16 и более процессорами на одном кристалле, а не повышению производительности микропроцессоров с одним ядром. Следовательно, в наши дни все большее распространение получают многоядерные настольные компьютеры и даже многоядерные встраиваемые устройства. Вычислительная мощность этих машин увеличивается не за счет более быстрого выполнения одной задачи, а в результате параллельного выполнения сразу нескольких задач. В прошлом программисты могли сидеть сложа руки и смотреть, как без каких-либо усилий с их стороны с каждым новым поколением процессоров созданные ими программы работают быстрее. Но теперь, как сказал Херб Саттер (Herb Sutter), «бесплатных обедов больше не будет». Чтобы программы могли воспользоваться преимуществами увеличенной вычислительной мощности, их следует разрабатывать с прицелом на одновременное выполнение сразу нескольких задач. Поэтому программистам нужно принять это к сведению, а тем, кто до сих пор игнорировал конкурентность, — стремиться добавить ее в свой арсенал.

Существует два способа применения конкурентности для повышения производительности. Первый и наиболее очевидный — разбить одну задачу на части и выполнять их параллельно, сокращая тем самым общее время выполнения. Это *параллелизм задач*. На словах все довольно просто, но процесс может оказаться совсем не простым, потому что, возможно, между различными частями программы существует множество зависимостей. Разбиение на части возможно либо относительно обработки: один поток выполняет одну часть алгоритма, другой поток — другую, либо относительно данных: все потоки выполняют одну и ту же операцию с разными частями данных. Последний подход называется *параллелизмом данных*.

Алгоритмы, легко поддающиеся параллелизму, зачастую называют *сугубо параллельными*. Не стоит смущаться легкости распараллеливания такого кода, в ней нет ничего плохого. Мне встречались и другие названия таких алгоритмов: *естественно параллельные* и *удобно параллельные*. Сугубо параллельные алгоритмы обладают высокой масштабируемостью — по мере увеличения количества доступных аппаратных потоков параллелизм в алгоритме может быть соответственно увеличен. Подобный алгоритм идеально характеризуется изречением: «Одна голова хорошо,

а две — лучшие». Те его части, которые не подпадают под определение сугубо параллельных, можно разделить на фиксированное (а следовательно, немасштабируемое) количество параллельно выполняемых задач. Методы разбиения задач на потоки рассматриваются в главах 8 и 10.

Второй способ применения конкурентности для повышения производительности — воспользоваться доступным параллелизмом для решения более сложных задач. Например, в зависимости от ситуации одновременно обрабатывать не один, а два, десять или двадцать файлов. Это не что иное, как *параллелизм данных*, то есть одновременное выполнение одной и той же операции с несколькими наборами данных. Есть и другой фокус. Обработка одного фрагмента данных по-прежнему занимает одно и то же время, но теперь в течение него можно обрабатывать больше данных. Очевидно, что у такого подхода есть ограничения и он не всегда будет приносить выгоду, но обусловленное им повышение пропускной способности может открыть новые возможности, например увеличится разрешение при обработке видео, если различные области изображения будут обрабатываться параллельно.

1.2.3. Когда не нужна конкурентность

Понимать, когда конкурентностью пользоваться не стоит, не менее важно. По сути, единственной причиной отказа от ее использования является отрицательный баланс выгод и затрат. Код, реализующий конкурентность, зачастую сложнее понять, поэтому создание и поддержка многопоточного кода сопряжены с прямыми интеллектуальными затратами, а дополнительная сложность может вызвать увеличение количества ошибок. Если потенциальный прирост производительности недостаточно велик или разбиение задач выражено нечетко и это не может оправдать дополнительные затраты времени на разработку, необходимые для достижения успеха, а также дополнительные затраты, связанные с поддержкой многопоточного кода, использовать конкурентность не имеет смысла.

Кроме того, прирост производительности может не оправдать ожиданий, ведь с запуском потока связаны некоторые издержки, поскольку операционная система должна выделить соответствующие ресурсы ядра и пространство стека, а затем добавить новый поток в диспетчер потоков, а на все это нужно время. Если выполняемая в потоке задача завершается довольно быстро, то время, затрачиваемое на саму задачу, может быть затенено издержками на запуск потока, что способно ухудшить общую производительность приложения, по сравнению с выполнением задачи в том потоке, который породил поток конкретно под нее.

Кроме того, потоки являются ограниченным ресурсом. Одновременный запуск нескольких потоков потребляет ресурсы операционной системы и может замедлить работу системы в целом. При этом использование слишком большого количества потоков может исчерпать доступную память или адресное пространство процесса, поскольку каждый поток требует отдельного пространства стека. Это особенно актуально для 32-разрядных процессов с плоской архитектурой, где имеется ограничение 4 Гбайт доступного адресного пространства. Если каждый поток имеет

стек 1 Мбайт (во многих системах именно так), то адресное пространство будет использоваться с 4096 потоками, не оставляя места для кода, статических данных или данных кучи. Хотя 64-разрядные или более крупные системы не имеют данного прямого ограничения адресного пространства, их ресурсы по-прежнему ограничены: если запускается слишком много потоков, то в конечном итоге возникают проблемы. Несмотря на то что для ограничения количества потоков можно воспользоваться пулами потоков (см. главу 9), панацеей они не являются, так как и у них есть проблемы.

Если серверная часть клиент-серверного приложения запускает отдельный поток для каждого подключения, то для небольшого числа подключений будет обеспечена нормальная работа. Но если этот метод используется для сервера запросов, обрабатывающего множество подключений, то при запуске слишком большого количества потоков весьма высока вероятность исчерпания системных ресурсов. В таком случае обеспечить оптимальную производительность поможет аккуратное использование пулов потоков (см. главу 9).

И наконец, чем больше потоков запущено, тем больше переключений контекста должна выполнять операционная система. Каждое такое переключение занимает время, которое можно было бы потратить на полезную работу, поэтому в какой-то момент добавление дополнительного потока снизит общую производительность приложения, а не увеличит ее. Следовательно, при попытке получить максимально возможную производительность системы необходимо настроить количество запускаемых потоков, чтобы учесть доступный аппаратный параллелизм или его отсутствие.

Использование конкурентности для повышения производительности похоже на любую другую стратегию оптимизации: оно может существенно повысить производительность приложения, но способно и усложнить код, затруднить его понимание и повысить вероятность ошибок. Поэтому применять конкурентность стоит только для тех критически важных для производительности частей приложения, где можно получить ощутимую выгоду. Конечно, если потенциал прироста производительности менее важен, чем ясность конструкции или разделение задач, все же стоит воспользоваться многопоточной конструкцией.

Предположим, вы уже решили распараллелить свое приложение либо для повышения производительности, либо для разделения задач, либо просто «в рамках кампании по применению многопоточности». Что это будет означать для программистов на C++?

1.3. Конкурентность и многопоточность в C++

Стандартная поддержка конкурентности с помощью многопоточности относительно нова для C++. Возможность создать многопоточный код, не прибегая к расширениям для конкретной платформы, появилась только с выходом стандарта C++11. Чтобы понять, чем обоснованы многие решения в стандартной библиотеке потоков C++, важно знать историю их появления.

1.3.1. История поддержки многопоточности в C++

В стандарте C++ 1998 года существование потоков не признается, а действия различных элементов языка описываются в понятиях последовательной абстрактной машины. Более того, модель памяти формально не определена, поэтому создавать многопоточные приложения без характерных для компилятора расширений стандарта C++ 1998 года невозможно.

Поставщики компилятора могут свободно добавлять расширения языка, а распространённость API языка C для многопоточности, например, в стандарте POSIX C и API Windows компании Microsoft побудила многих поставщиков компилятора C++ поддерживать многопоточность с различными расширениями, специфичными для конкретных платформ. Эта поддержка компилятора, как правило, ограничивается при наличии нескольких потоков разрешением использовать соответствующий API языка C для платформы и обеспечением работы библиотеки времени выполнения C++ (например, кода для механизма обработки исключений). Хотя формальная многопоточная модель памяти была предоставлена немногими поставщиками компиляторов, поведение компиляторов и процессоров позволяло создать множество многопоточных программ на C++.

Не удовлетворившись применением платформенно-зависимых API языка C для работы с многопоточностью, программисты на C++ захотели, чтобы в используемых ими библиотеках классов были реализованы объектно-ориентированные средства для написания многопоточных программ. Среды разработки, например MFC, и универсальные библиотеки C++, такие как Boost и ACE, накопили в себе наборы классов C++, в которые заключались базовые API для конкретных платформ и которые предоставляли высокоуровневые средства, упрощающие решение задач применения многопоточности. Хотя конкретные детали библиотек классов существенно различаются, особенно в области запуска новых потоков, в целом форма классов имеет много общего. Одной из важнейших конструктивных особенностей, общей для многих библиотек классов C++ и дающей программисту существенные преимущества, является использование способа Resource Acquisition Is Initialization (RAII) (получение ресурса через инициализацию с блокировкой), чтобы гарантировать разблокировку мьютексов при выходе из соответствующей области.

Во многих случаях поддержка многопоточности существующих компиляторов C++ в сочетании с доступностью API для конкретных платформ и библиотек классов, не зависящих от конкретной платформы, таких как Boost и ACE, обеспечивает прочную основу для написания многопоточного кода на C++. Как следствие, возможно, существуют миллионы строк кода на C++, написанных в качестве части многопоточных приложений. Но отсутствие стандартной поддержки означает, что в ряде случаев при отсутствии модели памяти с поддержкой потоков у программистов возникают проблемы. Особенно у тех, кто пытается повысить производительность, используя знания об аппаратных особенностях процессора, или пишет кросс-платформенный код, в котором поведение компиляторов варьируется в зависимости от используемой платформы.

1.3.2. Поддержка конкурентности в стандарте C++11

С выпуском стандарта C++11 все изменилось. В нем есть не только модель памяти, ориентированная на использование многопоточности, но и стандартная библиотека C++, включающая классы для управления потоками (см. главу 2), защиту совместно используемых данных (см. главу 3), синхронизацию операций между потоками (см. главу 4) и низкоуровневые атомарные операции (см. главу 5).

Библиотека потоков C++11 базируется на предыдущем опыте, накопленном благодаря использованию упомянутых ранее библиотек классов C++. В частности, в качестве модели, на которой основана новая библиотека, действовала библиотека потоков Boost, причем имена и структуры многих классов перекликались с именами и структурами соответствующих классов из Boost. Развитие стандарта превратило этот процесс в двусторонний, в рамках которого изменялась и сама библиотека потоков Boost, чтобы во многих отношениях соответствовать стандарту C++. Поэтому пользователям, ранее применявшим Boost, многое должно быть уже знакомо.

Как упоминалось в начале главы, с выходом стандарта C++11 в язык было внесено множество улучшений, облегчающих жизнь программистов, и поддержка конкурентности — одно из них. Хотя другие изменения зачастую не относятся к тематике данной книги, некоторые из них непосредственно повлияли на библиотеку потоков и способы ее использования. Краткий обзор этих функций языка дается в приложении А.

1.3.3. Расширение поддержки конкурентности и параллелизма в C++14 и C++17

Единственным добавлением в C++14, касавшимся конкретной поддержки конкурентности и параллелизма, стал новый тип мьютекса для защиты совместно используемых данных (см. главу 3). С выходом C++17 новинок стало значительно больше: для начинающих появился полный набор параллельных алгоритмов (см. главу 10). Оба стандарта улучшили основной язык и стандартную библиотеку в целом, и эти улучшения могут существенно упростить создание многопоточного кода.

Как уже упоминалось, для конкурентности имеется техническая спецификация, в которой дается описание расширений функций и классов, предоставляемых стандартом C++, особенно имеющих отношение к синхронизации операций между потоками (см. главу 4).

Поддержка непосредственно в C++ атомарных операций позволяет программистам создавать эффективный код с определенной семантикой, не требуя применения языка ассемблера конкретной платформы. Это настоящая находка для тех, кто стремится писать эффективный переносимый код, причем компилятор не только обращает внимание на специфику платформы, но и дает возможность создать оптимизатор, учитывающий семантику операций, что способно оптимизировать программу в целом.

1.3.4. Эффективность, обеспеченная библиотекой потоков C++

У разработчиков, занимающихся высокопроизводительными вычислениями, зачастую вызывает вопросы эффективность языка C++ в целом и, в частности, тех классов C++, которые служат оболочкой для низкоуровневых средств, имеющихся в новой стандартной библиотеке потоков C++. Стремясь достичь максимального уровня производительности, важно понимать различие между затратами на реализацию с применением любых средств высокого уровня и реализацию непосредственного использования базовых низкоуровневых средств. Эти затраты являются *платой за абстракцию*.

Специалисты Комитета по стандартизации C++ знали об этом, разрабатывая стандартную библиотеку C++ в целом и стандартную библиотеку потоков C++ в частности. Одной из целей проектирования библиотек было сведение практически к нулю выгоды от непосредственного использования низкоуровневых API там, где должно быть предоставлено аналогичное средство. Поэтому библиотека была разработана с прицелом на обеспечение эффективной реализации (и низкой платы за абстракцию) на большинстве основных платформ.

Другой целью Комитета по стандартизации C++ было добиться того, чтобы язык C++ обеспечивал достаточные возможности низкоуровневого программирования для всех, кто захочет для достижения максимальной производительности вести разработку с учетом конкретного оборудования. Поэтому наряду с новой моделью памяти поставляется и комплексная библиотека атомарных операций, допускающая непосредственное управление отдельными битами и байтами, а также синхронизацию между потоками и отображение любых изменений. Теперь эти атомарные типы и соответствующие операции можно задействовать во многих местах, где разработчики ранее предпочли бы перейти на язык ассемблера конкретной платформы. Код, использующий новые стандартные типы и операции, отличается большей степенью переносимости и простотой поддержки.

Стандартная библиотека C++ предоставляет также высокоуровневые абстракции и средства, которые облегчают создание многопоточного кода и уменьшают вероятность возникновения в нем ошибок. Порой использование этих средств приводит к снижению производительности, поскольку приходится выполнять дополнительный код. Но это не обязательно становится следствием более высокой платы за абстракцию. В целом ущерб не выше, чем при создании эквивалентных функциональных средств вручную, а встроить объемный дополнительный код компилятор может и в любом другом случае.

В ряде случаев высокоуровневые средства предоставляют дополнительные функциональные возможности, выходящие за рамки того, что может понадобиться в конкретном случае. Чаще всего это не вызывает никаких проблем: вы не несете особых расходов на то, что не используете. Но в редких случаях эти неиспользуемые функциональные возможности влияют на производительность другого кода. Когда нужен высокий уровень производительности, а стоимость применения стандартных средств слишком высока, возможно, стоит создать нужные функциональные сред-

ства вручную, собрав их из объектов более низкого уровня. В подавляющем большинстве случаев усложнение кода и вероятность увеличения количества ошибок существенно перевешивают потенциальные выгоды от незначительного прироста производительности. Даже если профилирование показывает, что узким местом являются средства стандартной библиотеки C++, это может быть связано с неудачной конструкцией приложения, а не с плохой реализацией библиотеки. Например, то, что за мьютекс конкурирует слишком много потоков, негативно влияет на производительность. Вероятно, выгоднее было бы не пытаться сэкономить незначительное время на операциях мьютекса, а реструктурировать приложение, чтобы уменьшить число конфликтов, связанных с мьютексом. Разработка приложений с прицелом на уменьшение количества конфликтов рассматривается в главе 8.

В тех редких случаях, когда стандартная библиотека C++ не обеспечивает нужные производительность или поведение, может потребоваться использовать средства, ориентированные на применение конкретной платформы.

1.3.5. Средства, ориентированные на использование конкретной платформы

Хотя библиотека потоков C++ дает довольно широкие возможности для применения многопоточности и конкурентности, на любой отдельно взятой платформе будут средства, зависящие только от нее и выходящие за рамки предлагаемого библиотекой. Чтобы обеспечить возможность получить легкий доступ к этим средствам, не отказываясь от преимуществ задействования стандартной библиотеки потоков C++, типы в библиотеке потоков C++ предусматривают компонентную функцию `native_handle()`, позволяющую работать на уровне API для конкретной платформы. По своей природе любые операции, выполняемые с помощью `native_handle()`, полностью зависят от платформы и выходят за рамки тематики данной книги (и самой стандартной библиотеки C++).

Прежде чем начать присматриваться к возможностям использования средств, учитывающих особенности конкретной платформы, важно понять, что именно обеспечивает стандартная библиотека, поэтому начнем с примера.

1.4. Приступаем к практической работе

Итак, у вас в наличии великодушный компилятор C++11/C++14/C++17. Что дальше? Как выглядит многопоточная программа на C++? Она похожа на любую другую программу на C++ с обычным сочетанием переменных, классов и функций. Единственное реальное отличие заключается в том, что некоторые функции могут выполняться одновременно, поэтому необходимо обеспечить безопасность совместно используемых данных при конкурентном доступе к ним в соответствии с инструкциями, изложенными в главе 3. Чтобы одновременно запустить функции для управления различными потоками, необходимо воспользоваться особыми функциями и объектами.

1.4.1. Здравствуй, мир конкурентности

Начнем с классического примера — программы для вывода фразы Hello World. Далее показана простая программа Hello World, которая выполняется в одном потоке и служит базой для перехода к использованию нескольких потоков:

```
#include <iostream>
int main()
{
    std::cout<<"Hello World\n";
}
```

В этой программе в стандартный поток вывода записывается фраза Hello World. Сравним ее с простой программой Hello Concurrent World, показанной в листинге 1.1 и запускающей для отображения сообщения отдельный поток.

Листинг 1.1. Простая программа Hello Concurrent World

```
#include <iostream>
#include <thread>
void hello()
{
    std::cout<<"Hello Concurrent World\n";
}
int main()
{
    std::thread t(hello);
    t.join();
}
```

Первым ее отличием является использование директивы `#include <thread>`. Объявления, предназначенные для поддержки многопоточности в стандартной библиотеке C++, представлены в новых заголовках: функции и классы для управления потоками объявляются в `<thread>`, а функции и классы для защиты совместно используемых данных — в других заголовках.

Второе отличие — переименование кода для записи сообщения в отдельную функцию. Это связано с тем, что каждый поток должен иметь *исходную функцию*, с которой начинается новый поток выполнения. Для начального потока в приложении это `main()`, а для любого другого потока такая функция указывается в конструкторе объекта `std::thread`. В данном случае это объект `std::thread` с именем `t`, имеющий в качестве исходной новую функцию `hello()`.

II в этом заключается следующее отличие: вместо непосредственной записи в стандартный вывод или вызова `hello()` из `main()` эта программа запускает для выполнения своей задачи новый поток, в результате чего потоков становится два — исходный поток, который начинается с `main()`, и новый, начинающийся с `hello()`.

После запуска нового потока исходный продолжает выполняться. Если бы он не ждал завершения нового потока, то запросто продолжил бы работу до конца `main()` и завершил программу, возможно, еще до того, как смог бы запуститься новый поток. Вот почему в соответствии с положениями, которые рассматриваются в главе 2, мы

добавили вызов `join()` — он заставляет вызывающий поток (`main()`) ожидать тот поток, что связан с объектом `std::thread`, — в данном случае поток `t`.

Если вам показалось, что для элементарного вывода сообщения в стандартный вывод работы слишком много, то так оно и есть, — в подразделе 1.2.3 выше мы говорили, что обычно для решения такой простой задачи нет смысла создавать несколько потоков, особенно если главному потоку в это время нечего делать. Далее в книге будут рассмотрены примеры сценариев, в которых выгода от применения нескольких потоков не вызывает никаких сомнений.

Резюме

В этой главе мы выяснили, что подразумевается под конкурентностью и многопоточностью, и определили причины, по которым стоит или не стоит использовать их в создаваемых вами приложениях. Была также рассмотрена история применения многопоточности в C++, начиная с полного отсутствия поддержки в стандарте 1998 года, появления в дальнейшем различных расширений для конкретных платформ и заканчивая надлежащей поддержкой многопоточности в стандарте C++11 и стандартах C++14 и C++17, а также выработкой технической спецификации по конкурентности. Эта поддержка оказалась весьма своевременной. Она позволила программистам воспользоваться преимуществами аппаратного параллелизма, которые стали доступны в современных процессорах, поскольку их производители пошли по пути наращивания мощности за счет реализации нескольких ядер, а не увеличивая быстродействие одного ядра.

Кроме этого, на примерах из раздела 1.4 вы увидели, как просто использовать классы и функции из стандартной библиотеки C++. Применение в C++ нескольких потоков не представляет особой сложности. Труднее разработать код, который будет вести себя именно так, как задумано.

После рассмотрения примеров из раздела 1.4 пришло время для чего-то более существенного. В главе 2 поговорим о классах и функциях, доступных для управления потоками.